

RE-GCN

MU-ty

3.22

1研究背景、问题定义

- 时序知识图谱TKG
 - 定义：随时间演变的知识图谱序列
 - 事实形式：(实体, 关系, 实体, 时间戳)
 - 关键特性
 - 同一时间戳内并发事实的结构依赖
 - 时序相邻事实的序列模式
 - 实体的静态属性 (类型、位置)
- 推理任务
 - 外推设置：预测未来时间戳的事实
 - 实体预测：预测 $(s, r, ?, t + 1)$ 或 $(?, r, o, t + 1)$
 - 关系预测：预测 $(s, ?, o, t + 1)$

1RE-GCN创新

- 3个关键要素
 - 结构依赖：同一时间戳内 KGs 的并发事实关联
 - 序列模式：时序相邻事实的演化趋势
 - 静态属性：实体类型、位置等背景知识
- 模型目标
 - 同时支持实体预测和关系预测
 - 高效建模完整历史 KG 序列
 - 避免为每个查询单独编码历史

2相关工作

- 图卷积网络（GCN）是一种结合图中内容特征和结构特征的代表性模型，一些研究将其推广到关系感知图卷积网络，以处理知识图谱。
- 时序知识图谱：插值和内推
- 本文聚焦**外推**设置：基于历史事实预测未来时间戳的新事实。
 - 部分相关模型用时点过程建模连续时间时序知识图谱（TKG），但与本文工作方向不同；Glean 依赖事件摘要文本，实际应用受限。
 - CyGNet、RE-NET 与本文最相关，均从单查询视角做实体预测，但忽略了不同时间戳图谱的结构依赖与实体静态属性。
 - 本文方法 RE-GCN 将知识图谱序列整体建模，兼顾全部结构依赖，效率与效果均有显著提升。

3问题定义

符号	描述
G, G^s, G_t	时序知识图谱 (TKG)、静态图、时间戳 t 处的知识图谱 (KG)
$\mathcal{V}, \mathcal{R}, \mathcal{E}_t$	TKG 中的实体集、关系集、事实集 (在 t 处)
$\mathcal{V}^s, \mathcal{R}^s, \mathcal{E}^s$	静态图中的实体集、关系集、边集
H_t, R_t	实体与关系的嵌入矩阵, 在 t 时刻
H, R, H^s	随机初始化的实体嵌入矩阵以及关系, 实体的静态嵌入矩阵

- 1、实体预测:给定查询“ $s, r, ?, t+1$ ”, RE-GCN根据主体实体 s 、关系 r 以及历史知识图谱序列 $G_{t-m+1:t}$, 对所有客体实体的条件概率向量进行建模:

$$\vec{p}(o|s, r, G_{t-m+1:t}) = \vec{p}(o|s, r, H_t, R_t)$$

- 2、关系预测: 给定查询“ $s, ?, o, t+1$ ”, RE-GCN模型根据主体实体 s 、客体实体 o 以及历史知识图谱序列 $G_{t-m+1:t}$ 对所有关系的条件概率向量进行建模, 给定如下:

$$\vec{p}(r|s, o, G_{t-m+1:t}) = \vec{p}(r|s, o, H_t, R_t)$$

RE-GCN模型

- 演化单元

- 演化单元由一个关系感知图卷积网络(GCN)、两个门控循环组件以及一个静态图约束组件组成。
- **关系感知GCN**旨在捕获每个时间戳下知识图谱(KG)内部的结构依赖关系
- **两个门控循环组件**以自回归方式对历史知识图谱序列进行建模
- **时间门控循环组件**和**GRU组件**分别获取每个时间戳下实体和关系的演化表示
- **静态图约束组件**通过在实体的静态嵌入和演化嵌入之间**添加约束**，将静态属性整合到演化嵌入中
- 演化单元计算从最近 m 个时间戳的知识图谱序列（即 $G_{t-m+1}, \dots, G_t$ ）到实体嵌入矩阵序列（ $H_{t-m+1}, \dots, H_t$ ）以及关系嵌入矩阵序列（ $R_{t-m+1}, \dots, R_t$ ）的循环映射

4.1.1 并发事实间的结构依赖

- 关系感知GCN $\left\{ \begin{array}{l} \omega \text{层图卷积网络} \\ \text{消息传递机制: 对象实体从主题实体 + 关系获取信息} \\ \text{公式: } \vec{h}_{o,t}^{l+1} = f\left(\frac{1}{c_o} \sum_{(s,r), \exists (s,r,o) \in \mathcal{E}_t} \mathbf{w}_1^l (\vec{h}_{s,t}^l + \vec{r}_t) + \mathbf{w}_2^l \vec{h}_{o,t}^l\right) \\ \text{捕捉: 同一时间戳内事实间的结构依赖} \end{array} \right.$

4.1.2跨时间相邻事实的序列模式

- 时间门控循环组件 {
 - 解决堆叠GCN的过平滑和梯度消失
 - 公式 {
 - 时间门: $U_t = \sigma(W_4 H_{t-1} + b)$
 - 实体更新: $H_t = U_t \otimes H_t^\omega + (1 - U_t) \otimes H_{t-1}$
 - 捕捉: 时序相邻事实的序列模型

4.1.3跨时间相邻事实的序列模式

- GRU关系更新 {
 - 对关系嵌入进行序列模式
 - 公式: $R_t = GRU(R_{t-1}, R'_t)$
 - 输入: 相关实体池化表示 + 关系自身嵌入

4.1.4静态图约束组件

- { 构建实体静态图（基于实体类型、位置等）
使用1层 $R - GCN$ 获取静态嵌入
角度约束：演化嵌入与静态嵌入夹角 $\leq \theta_x$ (随 x 递增)
公式8 - 12：损失函数 L_{st}
作用：提供实体背景知识，改善初始表示

组件 1: 关系感知 GCN (Relation-aware GCN)

—— 捕捉并发事实的结构依赖

- 建模单个时间戳 t 内 KG 的结构依赖: 包括实体间通过事实的关联 (如“政府 - 施压 - 民众”的间接影响)、关系间通过共享实体的关联 (如同一实体参与的多个关系的相互作用)
 - 步骤:
 - 网络配置: 采用 ω 层关系感知 GCN (实验中 YAGO 设为 1 层, 其他数据集设为 2 层), 每层使用 dropout (率 0.2) 防止过拟合。
 - 消息传递机制: 基于“实体 - 关系”的平移特性 ($\vec{h}_s + \vec{r}$), 通过聚合邻居实体信息更新当前实体嵌入, 公式 (3) 拆解

RReLU 激活函数 (缓解梯度消失, 适配多关系图的稀疏性)

$$\vec{h}_{o,t}^{l+1} = f \left(\frac{1}{c_o} \sum_{(s,r), \exists (s,r,o) \in \mathcal{E}_t} \mathbf{W}_1^l (\vec{h}_{s,t}^l + \vec{r}_t) + \mathbf{W}_2^l \vec{h}_{o,t}^l \right)$$

实体 o 在时间戳 t 第 l 层的嵌入

第 l 层的聚合权重和自环权重 (可学习参数)

归一化常数 (实体 o 的入度, 避免嵌入被高连接度实体主导)

组件 1：关系感知 GCN (Relation-aware GCN)

—— 捕捉并发事实的结构依赖

- 无事实实体处理：对于未参与任何事实的实体，仅执行自环操作（补充参数 W_{3I} ），视为“实体自演化”，避免嵌入更新中断
- 输出结果
 - 时间戳 t 经过 ω 层 GCN 后的实体嵌入矩阵 H_{ω}^t （已融合该时间戳内所有事实的结构依赖）

组件 2：门控循环组件 —— 捕捉跨时间戳的时序模式

- 建模跨时间戳的演化规律：**实体的行为趋势**（如“持续批评某类实体”）、**关系的出现模式**（如“访问后伴随合作”），同时解决深层 GCN 的**过平滑**和**梯度消失**问题
 - 时间门循环组件（Time Gate）—— 实体时序更新
 - GRU 组件 —— 关系时序更新

时间门循环组件 (Time Gate)

- 核心问题：直接堆叠 GCN 会导致实体嵌入趋同（过平滑），尤其当相邻时间戳存在重复关系时。
- 实现逻辑：通过门控机制动态融合“当前时间戳结构嵌入” (H_{ω}^t) 和“上一时间戳演化嵌入” (H_{t-1})

时间门权重矩阵 (sigmoid 激活输出 $[0,1]$, 0 表示完全保留历史, 1 表示完全更新为当前)

$$U_t = \sigma(W_4 H_{t-1} + b)$$

门控权重 (可学习), b 为偏置项

点积操作, 动态平衡 “当前结构信息” 和 “历史演化信息”

$$H_t = U_t \otimes H_{\omega}^t + (1 - U_t) \otimes H_{t-1}$$

GRU 组件 —— 关系时序更新

- 核心逻辑：关系的演化依赖其关联实体的演化（如“合作”关系的嵌入受参与合作的实体行为影响）

- ① 关联实体嵌入池化：对关系 r 在时间戳 t 关联的所有实体
($V_{r,t} = \{i \mid (i,r,o,t) \text{ 或 } (s,r,i,t) \in E_t\}$)，取其前一时间戳嵌入 H_{t-1} 的均值池化结果；
- ② GRU 输入构建：将池化结果与关系初始嵌入 r 拼接，公式（6）

$$\vec{r}'_t = [\text{pooling}(H_{t-1}, V_{r,t}); \vec{r}]$$

无对应事实的关系： $\vec{r}'_t = \vec{0}$
(避免嵌入更新紊乱)

- ③ 关系嵌入更新：通过 GRU 自回归建模时序模式，公式（7）

$$R_t = \text{GRU}(R_{t-1}, \vec{r}'_t)$$

所有关系的输入向量集合

约束：实体和关系嵌入的每行 L2 范数约束为 1（避免嵌入尺度不一致影响后续预测）

组件 3：静态图约束组件 (Static Graph Constraint)

—— 融合实体静态属性

- 将实体的静态属性（如类型、国家）作为背景知识融入演化嵌入，解决“历史数据稀疏时实体嵌入不准确”的问题。

- 步骤：

- ①**静态图构建**：基于实体名称字符串提取属性，构建静态图 $G_s=(V_s,R_s,E_s)$ ：

- 示例：实体“Police (Australia)”→ 添加关系“isA”（指向属性实体“Police”）和“country”（指向属性实体“Australia”）；
 - 支持的静态关系：实体类型（isA）、地理位置（country）、所属领域（domain）等。

- ②**静态嵌入计算**：采用 1 层无自环的 R-GCN（适配多关系静态图），公式 (8)

$$\vec{h}_i^s = \gamma \left(\frac{1}{c_i} \sum_{(r^s, j), \exists (i, r^s, j)) \in \mathcal{E}^s} \mathbf{W}_{r^s} \vec{h}_j^{ts}(j) \right)$$

实体i的静态嵌入

ReLU 激活函数

静态关系 r^s 的权重矩阵

随机初始化的静态输入嵌入

实体i在静态图中的连接数（归一化用）

组件 3：静态图约束组件 (Static Graph Constraint)

—— 融合实体静态属性

- ③演化嵌入约束：限制“实体演化嵌入”与“静态嵌入”的夹角不超过时间相关阈值，公式 (9) (10)

- 角度阈值： $\theta_x = \min(\gamma x, 90^\circ)$ ($x \in [0, m]$, $\gamma = 10^\circ$ 为角度递增步长)；时间越晚 (x 越大)，允许演化嵌入与静态嵌入的偏差越大 (实体行为可随历史扩展)；

- 约束损失：

$$L_x^{st} = \sum_{i=0}^{|\mathcal{V}|-1} \max\{\cos\theta_x - \cos(\vec{h}_i^s, \vec{h}_{t-m+x,i}), 0\}$$

- 总损失： $L^{st} = \sum_{x=0}^m L_x^{st}$

- 特殊处理：WIKI、YAGO、GDELT 数据集因缺少静态信息，仅在 ICEWS 系列数据集启用该组件。

4.2评分函数

- ConvTransE作为解码器
 - ConvTransE包含一个一维卷积层和一个全连接层
 - 实体概率向量 $\vec{p}(o|s, r, H_t, R_t) = \sigma(H_t \text{ConvTransE}(\vec{s}_t, \vec{r}_t))$
 - 关系概率向量 $\vec{p}(r|s, o, H_t, R_t) = \sigma(R_t \text{ConvTransE}(\vec{s}_t, \vec{o}_t))$

4.3多任务学习

- 联合损失=实体损失+关系损失+静态约束损失

$$L^e = \sum_{t=0}^{T-1} \sum_{(s,r,o,t+1) \in \mathcal{E}_{t+1}} \sum_{i=0}^{|\mathcal{V}|-1} y_{t+1,i}^e \log p_i(o|s, r, \mathbf{H}_t, \mathbf{R}_t)$$

$$L^r = \sum_{t=0}^{T-1} \sum_{(s,r,o,t+1) \in \mathcal{E}_{t+1}} \sum_{i=0}^{|\mathcal{R}|-1} y_{t+1,i}^r \log p_i(r|s, o, \mathbf{H}_t, \mathbf{R}_t),$$

实验设计与结果

- 数据集
 - ICEWS系列：危机预警系统(新闻事件)
 - 百科数据：WIKI,YAGO（实体知识）
 - 全球事件：GDELT（大规模事件数据）
- 评估指标
 - 采用MRR和 Hits@{1,3,10}作为实体预测和关系预测的评估指标
- 基线模型
 - 将RE-GCN模型与静态知识图谱推理模型及时间知识图谱（TKG）推理模型进行了对比

rgcn

rgcn/knowledge_graph.py

- 负责数据集读取和预处理
- 支持两类数据：实体分类数据和链路预测数据
- 时序知识图谱的数据通过 `load\_from\_local` 读取本地文件

rgcn/layers.py

- 定义图神经网络层
- 包括基础 RGCN 层、block 版本、以及 RE-GCN 中实际使用的 `UnionRGCNLayer`

rgcn/model.py

- 提供一个基础 RGCN 模型骨架 `BaseRGCN`
- 负责组装输入层、隐藏层、输出层

rgcn/utils.py

- 工具函数集中地
- 包括构图、评测、时间切分、过滤答案集、multi-step 推理辅助函数

src

src/main.py

- 程序入口
- 负责解析参数、加载数据、创建模型、训练、验证、测试、网格搜索

src/rrgcn.py

- 这是模型主体
- 定义 `RecurrentRGCN` 和 `RGCNCell`
- 把图卷积、关系演化、时间门控、静态约束、解码器串起来

src/decoder.py

- 定义打分函数
- `ConvTransE` 用于实体预测
- `ConvTransR` 用于关系预测

src/model.py

- 另一个基础 RGCN 骨架实现
- 供 `src/rrgcn.py` 里的 `RGCNCell` 继承使用

src/hyperparameter_range.py

- 定义网格搜索使用的超参数候选集合

实验设计与结果

- knowledge_graph.py

异构图（节点 = 实体，边 = 关系）

RGCNEntityDataset类（实体分类）

解析原始数据
→ BFS 剪枝 → 边
归一化 → 标签转
换

```
class RGCNEntityDataset(object):
    """RGCN Entity Classification dataset

    The dataset contains a graph depicting the connectivity of a knowledge
    base. Currently, four knowledge bases from the
    "RGCN paper <https://arxiv.org/pdf/1703.06103.pdf>" are supported: aifb,
    mutag, bgs, and am.

    The original knowledge base is stored as an RDF file, and this class will
    download and parse the RDF file, and performs preprocessing.

    An object of this class has 11 member attributes needed for entity
    classification:

    num_nodes: int
    | number of entities of knowledge base
    num_rels: int
    | number of relations (including reverse relation) of knowledge base
    num_classes: int
    | number of classes/labels that of entities in knowledge base
    edge_src: numpy.array
    | source node ids of all edges
    edge_dst: numpy.array
    | destination node ids of all edges
    edge_type: numpy.array
    | type of all edges
    edge_norm: numpy.array
    | normalization factor of all edges
    labels: numpy.array
    | labels of node entities
    train_idx: numpy.array
    | ids of entities used for training
    valid_idx: numpy.array
    | ids of entities used for validation
    test_idx: numpy.array
    | ids of entities used for testing

    Usually, users don't need to directly use this class. Instead, DGL provides
    wrapper function to load data (see example below).
    When loading data, besides specifying dataset name, user can provide two
    optional arguments:

    Parameters
    -----
    bfs_level: int
    | prune out nodes that are more than "bfs_level" hops away from
    | labeled nodes, i.e., nodes won't be touched during propagation. If set
    | to a number less or equal to 0, all nodes will be retained.
    relabel: bool
    | After pruning, whether or not to relabel all nodes with consecutive
    | node ids

    Examples
    -----
    Load aifb dataset, prune out nodes that are more than 3 hops away from
    labeled nodes, and relabel the remaining nodes with consecutive ids

    >>> from dgl.contrib.data import load_data
    >>> data = load_data(dataset='aifb', bfs_level=3, relabel=True)
    ...
```

```
def __init__(self, name):
    self.name = name
    self.dir = get_download_dir()
    tgz_path = os.path.join(self.dir, '{}.tgz'.format(self.name))
    download(download_prefix + '{}.tgz'.format(self.name), tgz_path)
    self.dir = os.path.join(self.dir, self.name)
    extract_archive(tgz_path, self.dir)

def load(self, bfs_level=2, relabel=False):
    self.num_nodes, edges, self.num_rels, self.labels, labeled_nodes_idx, self.train_idx, self.test_idx = _load_data(self.name, self.dir)

    # bfs to reduce edges
    if bfs_level > 0:
        print("removing nodes that are more than {} hops away".format(bfs_level))
        row, col, edge_type = edges.transpose()
        A = sp.csr_matrix((np.ones(len(row)), (row, col)), shape=(self.num_nodes, self.num_nodes))
        bfs_generator = _bfs_relational(A, labeled_nodes_idx)
        lvl = list()
        lvl.append(set(labeled_nodes_idx))
        for _ in range(bfs_level):
            lvl.append(next(bfs_generator))
        to_delete = list(set(range(self.num_nodes)) - set.union(*lvl))
        eid_to_delete = np.isin(row, to_delete) + np.isin(col, to_delete)
        eid_to_keep = np.logical_not(eid_to_delete)
        self.edge_src = row[eid_to_keep]
        self.edge_dst = col[eid_to_keep]
        self.edge_type = edge_type[eid_to_keep]

    if relabel:
        uniq_nodes, edges = np.unique((self.edge_src, self.edge_dst), return_inverse=True)
        self.edge_src, self.edge_dst = np.reshape(edges, (2, -1))
        node_map = np.zeros(self.num_nodes, dtype=int)
        self.num_nodes = len(uniq_nodes)
        node_map[uniq_nodes] = np.arange(self.num_nodes)
        self.labels = self.labels[uniq_nodes]
        self.train_idx = node_map[self.train_idx]
        self.test_idx = node_map[self.test_idx]
        print("{} nodes left".format(self.num_nodes))
    else:
        self.src, self.dst, self.edge_type = edges.transpose()

    # normalize by dst degree
    _, inverse_index, count = np.unique((self.edge_dst, self.edge_type), axis=1, return_inverse=True, return_counts=True)
    degrees = count[inverse_index]
    self.edge_norm = np.ones(len(self.edge_dst), dtype=np.float32) / degrees.astype(np.float32)

    # convert to pytorch label format
    self.num_classes = self.labels.shape[1]
    self.labels = np.argmax(self.labels, axis=1)
```

BFS剪枝

节点重编号

按目标节点+边类型分组，计算度数；
归一化因子 = 1 / 度数

边归一化

标签格式转换

RGCNLinkDataset类 (链接预测) 补全三元组

```
class RGCNLinkDataset(object):
    """RGCN link prediction dataset

    The dataset contains a graph depicting the connectivity of a knowledge
    base. Currently, the knowledge bases from the
    `RGCN paper <https://arxiv.org/pdf/1703.06103.pdf>`_ supported are
    FB15k-237, FB15k, wn18

    The original knowledge base is stored as an RDF file, and this class will
    download and parse the RDF file, and performs preprocessing.

    An object of this class has 5 member attributes needed for link
    prediction:

    num_nodes: int
    | number of entities of knowledge base
    num_rels: int
    | number of relations (including reverse relation) of knowledge base
    train: numpy.array
    | all relation triplets (src, rel, dst) for training
    valid: numpy.array
    | all relation triplets (src, rel, dst) for validation
    test: numpy.array
    | all relation triplets (src, rel, dst) for testing

    Usually, user don't need to directly use this class. Instead, DGL provides
    wrapper function to load data (see example below).

    Examples
    -----
    Load FB15k-237 dataset

    >>> from dgl.contrib.data import load_data
    >>> data = load_data(dataset='FB15k-237')

    """
```

读取三元组：将文本三元组转为(源节点ID, 关系ID, 目标节点ID)

```
def __init__(self, name, dir=None):
    self.name = name
    if dir:
        self.dir = dir
        self.dir = os.path.join(self.dir, self.name)
    else:
        self.dir = get_download_dir()
        tgz_path = os.path.join(self.dir, '{}.tar.gz'.format(self.name))
        download(_download_prefix + '{}.tgz'.format(self.name), tgz_path)
        self.dir = os.path.join(self.dir, self.name)
        extract_archive(tgz_path, self.dir)
    print(self.dir)

def load(self, load_time=True):
    entity_path = os.path.join(self.dir, 'entity2id.txt')
    relation_path = os.path.join(self.dir, 'relation2id.txt')
    train_path = os.path.join(self.dir, 'train.txt')
    valid_path = os.path.join(self.dir, 'valid.txt')
    test_path = os.path.join(self.dir, 'test.txt')
    entity_dict = _read_dictionary(entity_path)
    relation_dict = read_dictionary(relation_path)
    self.train = np.array(_read_triplets_as_list(train_path, entity_dict, relation_dict, load_time))
    self.valid = np.array(_read_triplets_as_list(valid_path, entity_dict, relation_dict, load_time))
    self.test = np.array(_read_triplets_as_list(test_path, entity_dict, relation_dict, load_time))
    self.num_nodes = len(entity_dict)
    print("# Sanity Check: entities: {}".format(self.num_nodes))
    self.num_rels = len(relation_dict)
    self.relation_dict = relation_dict
    self.entity_dict = entity_dict
    print("# Sanity Check: relations: {}".format(self.num_rels))
    print("# Sanity Check: edges: {}".format(len(self.train)))
```

文本实体/关系 → 唯一数字ID

layers.py

- DGL 所有图卷积都遵循三步：
 1. 消息函数 (msg_func) : 源节点向目标节点发送消息;
 2. 聚合函数 (fn.sum) : 目标节点把收到的所有消息求和;
 3. 更新函数 (apply_func) : 用聚合后的消息更新目标节点特征。

代码里的通用机制

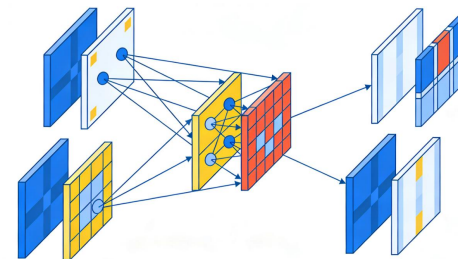
self_loop (自环) : 节点保留自身特征, 防止梯度消失;

skip_connect (残差连接) : 跨层传递特征, 深度网络不退化;

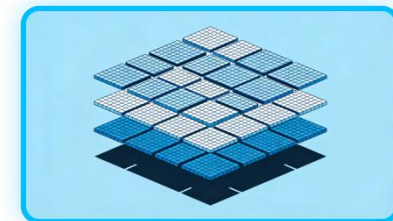
norm (归一化) : 数据集里的边归一化因子, 平衡节点度数;

xavier_uniform: 权重初始化, 保证神经网络训练稳定。

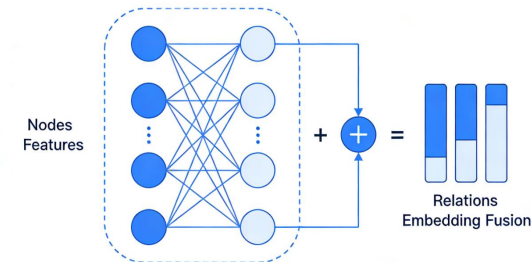
layers.py



- RGCNBasisLayer (经典RGCN: Basis^{线性组合生成权重}参数共享 → 实体分类)
- RGCNBlockLayer (RGCN变体: Block参数共享 → 大规模图)
- UnionRGCNLayer (改进RGCN: 融合关系嵌入 → 链接预测)



类名	参数共享方式	适用任务	对应数据集类
RGCNBasisLayer	Basis 基分解	实体分类	RGCNEntityDataset
RGCNBlockLayer	Block 块分解	实体分类	RGCNEntityDataset
UnionRGCNLayer	关系嵌入融合	链接预测	RGCNLinkDataset



- 初始化参数 → 计算自环/残差 → 消息传递 (关系卷积) → 聚合消息 → 更新节点特征 → 输出结果

关键代码

这个函数负责把一个 snapshot 的三元组转成 DGL 图

```
def build_sub_graph(num_nodes, num_rels, triples, use_cuda, gpu):
    """
    :param node_id: node id in the large graph
    :param num_rels: number of relation
    :param src: relabeled src id
    :param rel: original rel id
    :param dst: relabeled dst id
    :param use_cuda:
    :return:
    """

    def comp_deg_norm(g):
        in_deg = g.in_degrees(range(g.number_of_nodes())).float()
        in_deg[in_deg == 0].view(-1) = 1
        norm = 1.0 / in_deg
        return norm

    src, rel, dst = triples.transpose()
    src, dst = np.concatenate((src, dst)), np.concatenate((dst, src))
    rel = np.concatenate((rel, rel + num_rels))

    g = dgl.DGLGraph()
    g.add_nodes(num_nodes)
    g.add_edges(src, dst)
    norm = comp_deg_norm(g)
    node_id = torch.arange(0, num_nodes, dtype=torch.long).view(-1, 1)
    g.ndata.update({'id': node_id, 'norm': norm.view(-1, 1)})
    g.apply_edges(lambda edges: {'norm': edges.dst['norm'] * edges.src['norm']})
    g.edata['type'] = torch.LongTensor(rel)

    uniq_r, r_len, r_to_e = r2e(triples, num_rels)
    g.uniq_r = uniq_r
    g.r_to_e = r_to_e
    g.r_len = r_len
    if use_cuda:
        g = g.to(gpu)
        g.r_to_e = torch.from_numpy(np.array(r_to_e))
    return g
```

- 给每个三元组补一条逆边
- 给图中每个点计算入度归一化系数
- 把关系类型存到 edata['type']
- 生成 r_to_e, 表示每个关系涉及哪些实体

→ $\text{norm} = 1.0 / \text{in_deg}$

关键代码

时间演化主循环

1. 如果启用静态图，就先在静态图上更新实体表示，得到 static_emb
2. 遍历历史图 g_list
3. 对当前图中的每个关系，从相关实体聚合出关系输入特征
4. 用 GRUCell 更新关系嵌入 self.h_0
5. 用 RGCNCell 更新实体表示，并用时间门控融合历史与当前表示

$$gate = \sigma(HW + b)$$

$$H_{new} = gate \odot H_{current} + (1 - gate) \odot H_{old}$$

```
def forward(self, g_list, static_graph, use_cuda):
    gate_list = []
    degree_list = []

    if self.use_static:
        static_graph = static_graph.to(self.gpu)
        static_graph.ndata['h'] = torch.cat((self.dynamic_emb, self.words_emb), dim=0) # 演化得到的表示，和wordemb满足静态图约束
        self.static_rgcn_layer(static_graph, [])
        static_emb = static_graph.ndata.pop('h')[:self.num_ents, :]
        static_emb = F.normalize(static_emb) if self.layer_norm else static_emb
        self.h = static_emb
    else:
        self.h = F.normalize(self.dynamic_emb) if self.layer_norm else self.dynamic_emb[:, :]
        static_emb = None

    history_embs = []

    for i, g in enumerate(g_list):
        g = g.to(self.gpu)
        temp_e = self.h[g.r_to_e]
        x_input = torch.zeros(self.num_rels * 2, self.h_dim).float().cuda() if use_cuda else torch.zeros(self.num_rels * 2, self.h_dim).float()
        for span, r_idx in zip(g.r_len, g.uniq_r):
            x = temp_e[span[0]:span[1],:]
            x_mean = torch.mean(x, dim=0, keepdim=True)
            x_input[r_idx] = x_mean
        if i == 0:
            x_input = torch.cat((self.emb_rel, x_input), dim=1)
            self.h_0 = self.relation_cell_1(x_input, self.emb_rel) # 第1层输入
            self.h_0 = F.normalize(self.h_0) if self.layer_norm else self.h_0
        else:
            x_input = torch.cat((self.emb_rel, x_input), dim=1)
            self.h_0 = self.relation_cell_1(x_input, self.h_0) # 第2层输出==下一时刻第一层输入
            self.h_0 = F.normalize(self.h_0) if self.layer_norm else self.h_0
        current_h = self.rgcn.forward(g, self.h, [self.h_0, self.h_0])
        current_h = F.normalize(current_h) if self.layer_norm else current_h
        time_weight = F.sigmoid(torch.mm(self.h, self.time_gate_weight) + self.time_gate_bias)
        self.h = time_weight * current_h + (1-time_weight) * self.h
        history_embs.append(self.h)
    return history_embs, static_emb, self.h_0, gate_list, degree_list
```

关键代码

训练目标怎么组成

- 实体预测损失：用 ConvTransE
- 关系预测损失：用 ConvTransR
- 静态图损失：
 - 约束动态表示不要偏离静态图语义太多

组件3

```
class RecurrentRGCN(nn.Module):
    def get_loss(self, glist, triples, static_graph, use_cuda):
        :param glist:
        :param triplets:
        :param static_graph:
        :param use_cuda:
        :return:
        """
        loss_ent = torch.zeros(1).cuda().to(self.gpu) if use_cuda else torch.zeros(1)
        loss_rel = torch.zeros(1).cuda().to(self.gpu) if use_cuda else torch.zeros(1)
        loss_static = torch.zeros(1).cuda().to(self.gpu) if use_cuda else torch.zeros(1)

        inverse_triples = triples[:, [2, 1, 0]]
        inverse_triples[:, 1] = inverse_triples[:, 1] + self.num_rels
        all_triples = torch.cat([triples, inverse_triples])
        all_triples = all_triples.to(self.gpu)

        evolve_embs, static_emb, r_emb, _, _ = self.forward(glist, static_graph, use_cuda)
        pre_emb = F.normalize(evolve_embs[-1]) if self.layer_norm else evolve_embs[-1]

        if self.entity_prediction:
            scores_ob = self.decoder_ob.forward(pre_emb, r_emb, all_triples).view(-1, self.num_ents)
            loss_ent += self.loss_e(scores_ob, all_triples[:, 2])

        if self.relation_prediction:
            score_rel = self.rdecoder.forward(pre_emb, r_emb, all_triples, mode="train").view(-1, 2 * self.num_rels)
            loss_rel += self.loss_r(score_rel, all_triples[:, 1])

        if self.use_static:
            if self.discount == 1:
                for time_step, evolve_emb in enumerate(evolve_embs):
                    step = (self.angle * math.pi / 180) * (time_step + 1)
                    if self.layer_norm:
                        sim_matrix = torch.sum(static_emb * F.normalize(evolve_emb), dim=1)
                    else:
                        sim_matrix = torch.sum(static_emb * evolve_emb, dim=1)
                        c = torch.norm(static_emb, p=2, dim=1) * torch.norm(evolve_emb, p=2, dim=1)
                        sim_matrix = sim_matrix / c
                    mask = (math.cos(step) - sim_matrix) > 0
                    loss_static += self.weight * torch.sum(torch.masked_select(math.cos(step) - sim_matrix, mask))
            elif self.discount == 0:
                for time_step, evolve_emb in enumerate(evolve_embs):
                    step = (self.angle * math.pi / 180)
                    if self.layer_norm:
                        sim_matrix = torch.sum(static_emb * F.normalize(evolve_emb), dim=1)
                    else:
                        sim_matrix = torch.sum(static_emb * evolve_emb, dim=1)
                        c = torch.norm(static_emb, p=2, dim=1) * torch.norm(evolve_emb, p=2, dim=1)
                        sim_matrix = sim_matrix / c
                    mask = (math.cos(step) - sim_matrix) > 0
                    loss_static += self.weight * torch.sum(torch.masked_select(math.cos(step) - sim_matrix, mask))

        return loss_ent, loss_rel, loss_static
```

实验设计与结果

- 实验结果

- 实体预测
 - $ICEWS18: MRR\ 27.51\% (+1.34\%\ VS\ RE - NET)$
 - $WIKI: MRR\ 39.84\% (+8.97\%\ \uparrow)$
 - $YAGO: MRR\ 58.27\% (+11.46\%\ \uparrow)$
 - $GDELT$ 性能略低于 $RE - NET$ (抽象概念问题)
 - 关系预测
 - $ICE18: MRR\ 39.48\%$ (优于所有基线)
 - $WIKI: MRR\ 95.63\%$ (接近完美)
 - $YAGO: MRR\ 95.18\%$ (接近完美)
 - 关系数少使得任务相对容易
 - 效率对比
 - 加速比: $66 \times (ICE18)$, $82 \times (WIKI/YAGO)$, $17 \times (ICE05 - 15)$
 - 原因: 并行处理所有查询 vs $RE - NET$ 逐个处理

实验设计与结果

- 消融实验

- | | | |
|---|------------------|--|
| { | 演化单元 (- EE) | : 性能大幅下降 |
| | 静态图约束 (- st) | $\begin{cases} ICWS \text{ 系列性能下降, 验证静态信息价值} \\ WIKI/YAGO/GDELT \text{ 无此组件 (数据缺失)} \end{cases}$ |
| | 时间门控 (- tg) | $\begin{cases} \text{长序列时性能急剧下降} \\ \text{证明时间门对解决过平滑/梯度消失问题} \end{cases}$ |
| { | 评分函数 (+ FCN) | $\begin{cases} \text{替换 } ConvTransE \text{ 为 } FCN, \text{ 性能略低但仍然强} \\ \text{验证 } GCN \text{ 与卷积评分函数的兼容性} \end{cases}$ |

详细分析

- 实体预测
 - 邻居数越多性能下降
 - Seen实体（有历史交互）表现优于unseen
 - 无历史实体也能预测（静态图提供背景知识）
- 关系预测
 - 关系数越多性能下降
 - 重复事实占一定比例，验证时间门的必要性

图神经网络

- 核心原理：消息传递机制

收集消息→更新表示→多轮迭代

$$h^{k+1u} = \sigma(\sum(W_r \cdot h_v^k + b_r) + W_o \cdot h^{ku})$$

图卷积神经网络GCN

- **核心贡献：**将谱域卷积简化为空间域局部聚合，建立了深度学习在图结构上的基础范式。
- **局限：**仅适用于无向图，且默认邻居节点同等重要，忽略边类型。

关系感知GNN：关系图卷积网络(RGCN)

- 在处理知识图谱等多关系图数据时，传统GCN难以区分不同关系的影响。RGCN通过引入关系特定的变换和参数共享机制，有效解决了这一问题，成为处理异构图数据的经典模型。
 - 为每种关系类型 r 分配独立的权重矩阵 W_r ，精准建模不同关系对节点特征的影响。
 - 通过基分解 (Basis Decomposition) 或块分解减少参数量，防止模型过拟合。
 - 显式添加自环边和反向关系边，增强模型对有向图结构的表达能力。

RGCNBasisLayer(基分解) / RGCNBlockLayer(块分解) / UnionRGCNLayer

数据准备与模型训练阶段

数据准备阶段 (Data Preparation)

数据集加载：使用 RGCNLinkDataset 加载图谱

图构建：执行 build_sub_graph 构建子图结构

答案预处理：load_all_answers 加载真实标签



模型训练阶段 (Model Training)

模型初始化：实例化 BaseRGCN 基础模型

前向传播：逐层执行 RGCN 卷积计算节点嵌入

损失计算：采用负采样损失优化模型参数

模型评估阶段

预测分数计算

模型输出所有实体的预测分数矩阵，为后续评估提供数据基础。

答案过滤

调用 `filter_score()` 屏蔽已知真实答案，避免高估模型实际性能。

排名计算

执行 `sort_and_rank()` 对分数排序，精确计算真实答案在预测结果中的排名。

指标统计

通过 `stat_ranks()` 输出 MRR、Hits@k 等关键指标，完成模型性能评估。

知识图谱领域应用

链接预测 (Link Prediction)

核心任务：预测知识图谱中缺失的三元组 (h, r, t) ，是 RGCN 的关键应用场景。

评估指标：MRR (平均倒数排名)、Hits@1 / Hits@3 / Hits@10

代码对应：`get_total_rank()`，`filter_score()`

实体分类

对图谱中的实体进行类型预判与归类。例如：识别“姚明”的实体类型为“运动员”。

关系抽取

从非结构化文本中自动提取实体间的语义关系，用于构建或补全知识图谱网络。